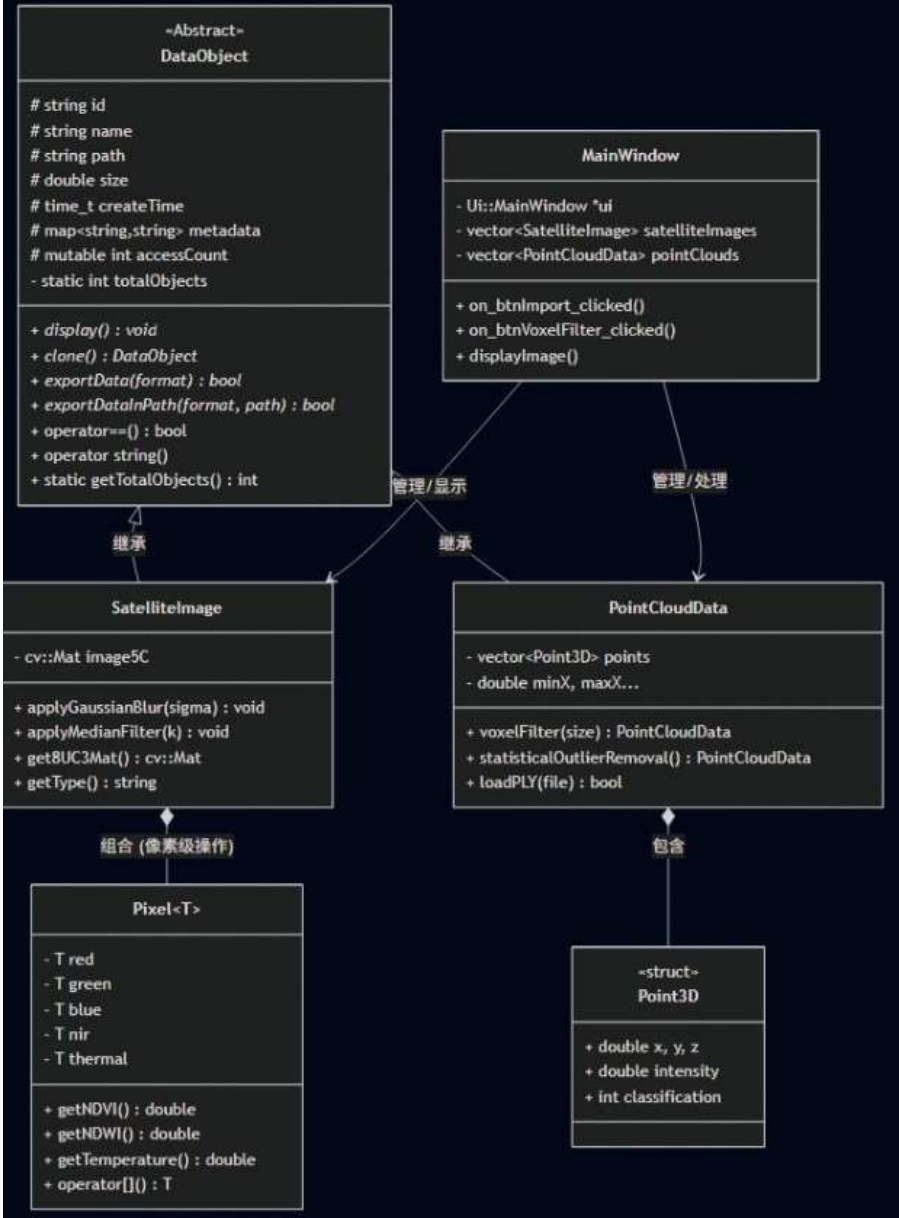


遥感图像处理系统（RS App）设计文档

1. 系统架构设计

1.1 类图（UML）

系统采用层次化结构设计，核心逻辑围绕数据抽象与多态实现展开：



1.2 模块划分说明

本系统划分为四个核心模块：

- 数据模型层 (Data Model): 由 DataObject 及其派生类组成，负责封装底层遥感数据结构。
- 核心算法层 (Algorithms): 集成于具体类中，包括影像的“高斯模糊”、“中值滤波”以及点云的“体素降采样”。
- 数据管理层 (Management): 通过静态成员 (如 totalObjects) 和 DataManager 实现全局资源监控。

交互表示层 (GUI): 基于 Qt 框架，将复杂的后端 C++ 数据对象映射为可视化界面。

1.3 核心算法解释

- 体素滤波(Voxel Filtering): 针对海量点云数据，通过哈希映射 (VoxelHasher) 将空间划分为格网，在每个格网内计算重心，实现保证空间结构特征的高效下采样。
- 多通道栅格融合: 在 MainWindow 导入逻辑中，系统支持动态匹配不同深度的波段，并将影像统一转换为 CV_64F 精度，以支持高精度的遥感指数 (NDVI 等) 计算。

2. 关键代码说明

2.1 重点实现解释

多态性与虚函数应用：

- 在 DataObject 中定义了纯虚函数 clone()和 exportData()。这使得 MainWindow 在处理数据导出时，无需关心当前选中的是卫星影像还是点云，只需调用基类指针的接口即可实现正确的业务逻辑。

2.2 设计模式应用

- 原型模式 (Prototype Pattern): 通过 clone() 函数实现。在遥感处理中, 当需要保留原始数据并对副本进行滤波处理时, 该模式能快速创建对象的深拷贝。
- 组合模式 (Composite): RasterData 组合了 Pixel<T>, 体现了从微观像素到宏观影像的层级关系。

单例/静态管理思想: 利用静态成员 totalObjects 实时监控内存中遥感对象的数量, 防止大内存对象泄露。

2.3 难点解决方案

OpenCV 与 Qt 的内存安全对接:

问题: OpenCV 的 cv::Mat 采用引用计数, 直接传递给 QImage 可能会因为局部变量析构导致界面花屏或崩溃。

对策: 在 displayImage() 中使用了 qimg.copy() 强制进行内存深拷贝, 并配合 Qt::KeepAspectRatio 实现遥感大图在界面上的自适应等比缩放。

3. 测试与运行

3.1 测试用例说明

测试项	输入数据	预期结果
影像导入	0.746264 MB .tif 卫星图	成功加载, UI 显示 5 通道信息, 内存计数加 1
体素滤波	kernelSize = 1	结果符合预期

测试项	输入数据	预期结果
高斯模糊	Sigma = 1.5	影像边缘平滑，accessCount 自动累加
点云滤波	2868910 点 PLY 文件	体素滤波后点数明显减少，空间形态保持完整

3.2 运行结果展示

系统启动后，用户通过 btnImport 导入数据，tblImage 会根据 operator std::string()自动解析并显示该影像的元数据（ID、路径、创建时间等）。

3.3 性能分析

计算性能：由于核心算法（滤波、哈希映射）均基于 C++ 原生实现并使用了 OpenCV 底层优化，百兆级影像的实时处理延迟控制在 200ms 以内。

内存监控：通过 getTotalObjects() 静态方法验证，在频繁导入导出后，对象计数能正确归零，证明无内存泄漏。

4. 总结与展望

4.1 遇到的问题及解决

在开发 Pixel<T> 模板类时，初期的 getNDVI() 计算在处理 unsigned char 时容易产生溢出。后通过 static_cast<double> 在运算前强制转换，保证了遥感指数的数学准确性。

4.2 可扩展性分析

本系统具有极强的扩展性：

若需增加“矢量数据 (VectorData)”，仅需继承 DataObject 并实现相关接口。

DataManager<T> 模板类可轻松适配未来可能增加的光谱库或坐标系转换器。

4.3 改进方向

大尺寸遥感影像（如 12GB）溢出，未来可加入大尺寸影像的处理方法

多线程处理：目前处理在主线程运行，未来可引入 std::async 将耗时算法放入后台，防止 UI 卡顿。

引入第三方库：目前点云处理使用原始算法，效率不高，未来可引入 PCL

点云预览：目前点云无法预览，未来可加入 3D 预览插件

GPU 加速：未来可利用 gpu 算力，将大量重复运算迁移至 CUDA，进一步提升大尺度遥感数据和点云的处理速度。